

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT
on

OPERATING SYSTEMS

Submitted by

VAIBHAV CHABBI(1BF24CS327)

in partial fulfilment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by **Vaibhav Chabbi(1BF24CS327)**, who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026

. The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Dr. Seema Patil
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1.	Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time. →FCFS → SJF (pre-emptive & Non-preemptive)	1-8
2.	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. → Priority (pre-emptive & Non-pre-emptive) →Round Robin (Experiment with different quantum sizes for RR algorithm)	9-17
3.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	18-21
4.	Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling	22-27
5.	Write a C program to simulate producer-consumer problem using semaphores	28-29
6.	Write a C program to simulate the concept of Dining Philosophers problem.	30-32
7.	Write a C program to simulate Bankers algorithm for the purpose of deadlock avoidance.	33-35

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.

C04	Conduct practical experiments to implement the functionalities of Operating system.
-----	---

Program -1

Question:

Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time.

→FCFS

→ SJF (pre-emptive & Non-preemptive)

Code:

=>FCFS:

```
#include <stdio.h>

int main() {
    int n, i, j;
    int arrival[20], burst[20];
    int completion[20], turnaround[20], waiting[20];
    int process[20];
    int current_time = 0;
    float avg_tat = 0, avg_wt = 0;
    printf("Enter total number of processes: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++) {
        process[i] = i + 1;
        printf("Process [%d]\n", i + 1);
        printf("Arrival time: ");
        scanf("%d", &arrival[i]);
        printf("Burst time: ");
        scanf("%d", &burst[i]);
    }
    for(i = 0; i < n - 1; i++) {
        for(j = i + 1; j < n; j++) {
            if(arrival[i] > arrival[j]) {
                int temp;
```

```

        temp = arrival[i];
        arrival[i] = arrival[j];
        arrival[j] = temp;
        temp = burst[i];
        burst[i] = burst[j];
        burst[j] = temp;
        temp = process[i];
        process[i] = process[j];
        process[j] = temp;
    }
}
}
// Calculate times
for(i = 0; i < n; i++) {
    if(current_time < arrival[i]) {
        current_time = arrival[i];
    }
    completion[i] = current_time + burst[i];
    turnaround[i] = completion[i] - arrival[i];
    waiting[i] = turnaround[i] - burst[i];
    current_time = completion[i];
    avg_tat += turnaround[i];
    avg_wt += waiting[i];
}
avg_tat /= n;
avg_wt /= n;
// Output
printf("\n Process Arrival Time Burst Time Completion Time Turnaround Time Waiting
Time\n");
for(i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
        process[i],

```

```

        arrival[i],
        burst[i],
        completion[i],
        turnaround[i],
        waiting[i]);
    }
    printf("\n Average Turnaround Time: %.2f\n", avg_tat);
    printf("Average Waiting Time: %.2f\n", avg_wt);
    return 0;
}

```

Output:

```

Enter total number of processes: 4
Process [1]
Arrival time: 0
Burst time: 2
Process [2]
Arrival time: 1
Burst time: 2
Process [3]
Arrival time: 5
Burst time: 3
Process [4]
Arrival time: 6
Burst time: 4

Process Arrival Time Burst Time Completion Time Turnaround Time Waiting Time
1         0         2         2             2             0
2         1         2         4             3             1
3         5         3         8             3             0
4         6         4        12             6             2

Average Turnaround Time: 3.50
Average Waiting Time: 0.75
1BF24CS327

```

=>SJF(Non-preemptive):

```
#include <stdio.h>
```

```

struct Process {
    int id;
    int arrival;
    int burst;
    int completion;
    int turnaround;
}

```

```

    int waiting;
    int finished;
};

int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    struct Process p[n];
    for (int i = 0; i < n; i++) {
        p[i].id = i + 1;
        printf("Enter Arrival Time and Burst Time for P%d: ", i + 1);
        scanf("%d %d", &p[i].arrival, &p[i].burst);
        p[i].finished = 0;
    }
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (p[i].arrival > p[j].arrival) {
                struct Process temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
    int current_time = 0, completed = 0;
    while (completed < n) {
        int idx = -1;
        int min_burst = 999999;
        for (int i = 0; i < n; i++) {
            if (!p[i].finished && p[i].arrival <= current_time) {
                if (p[i].burst < min_burst) {
                    min_burst = p[i].burst;
                    idx = i;
                }
            }
        }
        if (idx != -1) {
            p[idx].finished = 1;
            completed++;
            current_time += p[idx].burst;
        }
    }
}

```

```

        }
    }
}
if (idx == -1) {
    current_time++;
} else {
    p[idx].completion = current_time + p[idx].burst;
    p[idx].turnaround = p[idx].completion - p[idx].arrival;
    p[idx].waiting = p[idx].turnaround - p[idx].burst;
    current_time = p[idx].completion;
    p[idx].finished = 1;
    completed++;
}
}
printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].id, p[i].arrival, p[i].burst,
        p[i].completion, p[i].turnaround, p[i].waiting);
}
printf("1BF24CS327");
return 0;
}

```

Output:

```

Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 0 7
Enter Arrival Time and Burst Time for P2: 8 3
Enter Arrival Time and Burst Time for P3: 3 4
Enter Arrival Time and Burst Time for P4: 5 6

Process AT      BT      CT      TAT      WT
P1      0      7      7      7      0
P3      3      4      11     8      4
P4      5      6      20     15     9
P2      8      3      14     6      3
1BF24CS327

```

=>SJF(Pre-emptive):

```
#include <stdio.h>
```

```
struct Process {
```

```
    int pid;
```

```
    int at;
```

```
    int bt;
```

```
    int rt;
```

```
    int ct;
```

```
    int tat;
```

```
    int wt;
```

```
};
```

```
int main() {
```

```
    int n, i, current_time = 0, completed = 0, shortest;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    struct Process p[n];
```

```
    for(i = 0; i < n; i++) {
```

```
        p[i].pid = i + 1;
```

```
        printf("Enter Arrival Time and Burst Time for Process %d: ", p[i].pid);
```

```

scanf("%d %d", &p[i].at, &p[i].bt);
p[i].rt = p[i].bt;
}
while(completed < n) {
    shortest = -1;
    for(i = 0; i < n; i++) {
        if(p[i].at <= current_time && p[i].rt > 0) {
            if(shortest == -1 || p[i].rt < p[shortest].rt) {
                shortest = i;
            }
        }
    }
    if(shortest == -1) {
        current_time++;
    }
    else {
        p[shortest].rt--;
        current_time++;
        if(p[shortest].rt == 0) {
            completed++;
            p[shortest].ct = current_time;
            p[shortest].tat = p[shortest].ct - p[shortest].at;
            p[shortest].wt = p[shortest].tat - p[shortest].bt;
        }
    }
}
printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
for(i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);
}

```

```

float avg_tat = 0, avg_wt = 0;
for(i = 0; i < n; i++) {
    avg_tat += p[i].tat;
    avg_wt += p[i].wt;
}
avg_tat /= n;
avg_wt /= n;
printf("\nAverage Turnaround Time = %.2f", avg_tat);
printf("\nAverage Waiting Time = %.2f\n", avg_wt);
printf("1BF24CS327");
return 0;
}

```

Output:

```

Enter number of processes: 4
Enter Arrival Time and Burst Time for Process 1: 0 7
Enter Arrival Time and Burst Time for Process 2: 8 3
Enter Arrival Time and Burst Time for Process 3: 3 2
Enter Arrival Time and Burst Time for Process 4: 5 6

PID    AT    BT    CT    TAT    WT
P1      0     7     9     9     2
P2      8     3    12     4     1
P3      3     2     5     2     0
P4      5     6    18    13     7

Average Turnaround Time = 7.00
Average Waiting Time = 2.50
1BF24CS327

```

Program 2-

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

→ Priority (pre-emptive & Non-pre-emptive)

→ Round Robin (Experiment with different quantum sizes for RR algorithm)

Code:

=>Priority(Pre-emptive)

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
int main() {
```

```
    int n;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    int Process_ID[n], Arrival_Time[n], Burst_Time[n], Priority[n];
```

```
    int Remaining_Time[n], Completion_Time[n], Turnaround_Time[n], Waiting_Time[n];
```

```
    bool Finished[n];
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("\nEnter details for Process P%d\n", i + 1);
```

```
        Process_ID[i] = i + 1; // auto assign PID
```

```
        printf("Arrival Time: ");
```

```
        scanf("%d", &Arrival_Time[i]);
```

```
        printf("Burst Time: ");
```

```
        scanf("%d", &Burst_Time[i]);
```

```
        printf("Priority (lower number = higher priority): ");
```

```
        scanf("%d", &Priority[i]);
```

```
        Remaining_Time[i] = Burst_Time[i];
```

```
        Finished[i] = false;
```

```
    }
```

```
    int Current_Time = 0, Completed = 0;
```

```
    float totalTAT = 0, totalWT = 0;
```

```
    while (Completed < n) {
```

```

int Highest_Priority = 999999;
int Selected_Process = -1;
for (int i = 0; i < n; i++) {
    if (Arrival_Time[i] <= Current_Time && Remaining_Time[i] > 0) {
        if (Priority[i] < Highest_Priority) {
            Highest_Priority = Priority[i];
            Selected_Process = i;
        }
    }
}
if (Selected_Process == -1) {
    Current_Time++;
} else {
    Remaining_Time[Selected_Process]--;
    Current_Time++;
    if (Remaining_Time[Selected_Process] == 0) {
        Completion_Time[Selected_Process] = Current_Time;
        Turnaround_Time[Selected_Process] =
            Completion_Time[Selected_Process] - Arrival_Time[Selected_Process];
        Waiting_Time[Selected_Process] =
            Turnaround_Time[Selected_Process] - Burst_Time[Selected_Process];
        Finished[Selected_Process] = true;
        Completed++;
        totalTAT += Turnaround_Time[Selected_Process];
        totalWT += Waiting_Time[Selected_Process];
    }
}
}
printf("\nPID\tAT\tBT\tPriority\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t\t%d\t%d\t%d\n",

```

```

        Process_ID[i], Arrival_Time[i], Burst_Time[i],
        Priority[i], Completion_Time[i],
        Turnaround_Time[i], Waiting_Time[i]);
    }
    printf("\nAverage Turnaround Time: %.2f", totalTAT / n);
    printf("\nAverage Waiting Time: %.2f\n", totalWT / n);
    printf("USN:1BF24CS327");
    return 0;
}

```

Output:

```

Enter number of processes: 7

Enter details for Process P1
Arrival Time: 0
Burst Time: 8
Priority (lower number = higher priority): 3

Enter details for Process P2
Arrival Time: 1
Burst Time: 2
Priority (lower number = higher priority): 4

Enter details for Process P3
Arrival Time: 3
Burst Time: 4
Priority (lower number = higher priority): 4

Enter details for Process P4
Arrival Time: 4
Burst Time: 1
Priority (lower number = higher priority): 5

Enter details for Process P5
Arrival Time: 5
Burst Time: 6
Priority (lower number = higher priority): 2

Enter details for Process P6
Arrival Time: 6
Burst Time: 5
Priority (lower number = higher priority): 6

Enter details for Process P7
Arrival Time: 10
Burst Time: 1
Priority (lower number = higher priority): 1

PID    AT    BT    Priority    CT    TAT    WT
P1      0     8     3          15    15     7
P2      1     2     4          17    16    14
P3      3     4     4          21    18    14
P4      4     1     5          22    18    17
P5      5     6     2          12     7     1
P6      6     5     6          27    21    16
P7     10     1     1          11     1     0

Average Turnaround Time: 13.71
Average Waiting Time: 9.86
USN:1BF24CS327

```

=>Priority(Non-Preemptive)

```
#include <stdio.h>
```

```
struct Process {
```

```
    int pid;
```

```
    int at;
```

```
    int bt;
```

```
    int priority;
```

```
    int tat;
```

```
    int wt;
```

```
};
```

```
int main(){
```

```
    int n;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    struct Process p[n];
```

```
    for (int i = 0; i < n; i++) {
```

```
        p[i].pid=i+1;
```

```
        printf("Enter Arrival Time and Burst Time for process %d:", p[i].pid);
```

```
        scanf("%d %d",&p[i].at,&p[i].bt);
```

```
        printf("\nEnter priority for process %d:",p[i].pid);
```

```
        scanf("%d", &p[i].priority);
```

```
    }
```

```
    for (int i = 0; i < n - 1; i++) {
```

```
        for (int j = i + 1; j < n; j++) {
```

```
            if (p[i].priority < p[j].priority) {
```

```
                struct Process temp = p[i];
```

```
                p[i] = p[j];
```

```
                p[j] = temp;
```

```
            }
```

```
        }
```

```
    }
```

```

p[0].wt = 0;
p[0].tat = p[0].bt;
for (int i=1;i<n;i++){
    p[i].wt = p[i - 1].wt + p[i - 1].bt;
    p[i].tat = p[i].wt + p[i].bt;
}
printf("\nProcess\t Arrival Time\t Burst Time\t Priority\t Waiting Time\t Turnaround Time\n");
for (int i=0;i<n;i++){
    printf("%d\t%d\t\t%d\t\t%d\t\t%d\t\t%d\n",
p[i].pid,p[i].at,p[i].bt,p[i].priority,p[i].wt,p[i].tat);
}
float avgWT=0,avgTAT=0;
for (int i=0;i<n;i++) {
    avgWT += p[i].wt;
    avgTAT += p[i].tat;
}
printf("\nAverage Waiting Time: %.2f",avgWT/n);
printf("\nAverage Turnaround Time: %.2f\n",avgTAT/n);
printf("USN:1BF24CS327");
return 0;
}

```

Output:

```

Enter number of processes: 7
Enter Arrival Time and Burst Time for process 1:1 2

Enter priority for process 1:4
Enter Arrival Time and Burst Time for process 2:3 4

Enter priority for process 2:4
Enter Arrival Time and Burst Time for process 3:4 1

Enter priority for process 3:5
Enter Arrival Time and Burst Time for process 4:4 1

Enter priority for process 4:5
Enter Arrival Time and Burst Time for process 5:5 6

Enter priority for process 5:2
Enter Arrival Time and Burst Time for process 6:6 5

Enter priority for process 6:6
Enter Arrival Time and Burst Time for process 7:10 1

Enter priority for process 7:1

```

Process	Arrival Time	Burst Time	Priority	Waiting Time	Turnaround Time
6	6	5	6	0	5
4	4	1	5	5	6
3	4	1	5	6	7
2	3	4	4	7	11
1	1	2	4	11	13
5	5	6	2	13	19
7	10	1	1	19	20

```

Average Waiting Time: 8.71
Average Turnaround Time: 11.57
USN:1BF24CS327

```

=>Round Robin

```

#include <stdio.h>
#include <stdbool.h>
#define MAX 100

int main() {
    int n, time_quantum;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter time quantum: ");
    scanf("%d", &time_quantum);
    int process_id[MAX], arrival_time[MAX], burst_time[MAX];
    int remaining_time[MAX], completion_time[MAX];

```

```

int waiting_time[MAX], turnaround_time[MAX];
bool completed[MAX];
for (int i = 0; i < n; i++) {
    printf("\nEnter details for Process P%d\n", i + 1);
    process_id[i] = i + 1;
    printf("Arrival Time: ");
    scanf("%d", &arrival_time[i]);
    printf("Burst Time: ");
    scanf("%d", &burst_time[i]);
    remaining_time[i] = burst_time[i];
    completed[i] = false;
}
int current_time = 0, completed_count = 0;
float totalWT = 0, totalTAT = 0;
while (completed_count < n) {
    bool idle = true;
    for (int i = 0; i < n; i++) {
        if (arrival_time[i] <= current_time && remaining_time[i] > 0) {
            idle = false;
            if (remaining_time[i] > time_quantum) {
                remaining_time[i] -= time_quantum;
                current_time += time_quantum;
            } else {
                current_time += remaining_time[i];
                remaining_time[i] = 0;
                completion_time[i] = current_time;
                turnaround_time[i] = completion_time[i] - arrival_time[i];
                waiting_time[i] = turnaround_time[i] - burst_time[i];
                totalWT += waiting_time[i];
                totalTAT += turnaround_time[i];
                completed[i] = true;
            }
        }
    }
}

```

```

        completed_count++;
    }
}
}
if (idle) {
    current_time++;
}
}
printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        process_id[i], arrival_time[i], burst_time[i],
        completion_time[i], turnaround_time[i], waiting_time[i]);
}
printf("\nAverage Turnaround Time: %.2f", totalTAT / n);
printf("\nAverage Waiting Time: %.2f\n", totalWT / n);
printf("USN:1BF24CS327");
return 0;
}

```

Output:

```

Enter number of processes: 5
Enter time quantum: 2

Enter details for Process P1
Arrival Time: 0
Burst Time: 5

Enter details for Process P2
Arrival Time: 1
Burst Time: 3

Enter details for Process P3
Arrival Time: 2
Burst Time: 1

Enter details for Process P4
Arrival Time: 3
Burst Time: 2

Enter details for Process P5
Arrival Time: 4
Burst Time: 3

PID      AT      BT      CT      TAT      WT
P1        0        5      14      14        9
P2        1        3      12      11        8
P3        2        1        5        3        2
P4        3        2        7        4        2
P5        4        3      13        9        6

Average Turnaround Time: 8.20
Average Waiting Time: 5.40
USN:1BF24CS327

```

Program 3:

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

Code:

```
#include <stdio.h>
#include <string.h>
#define MAX 50
typedef struct Process{
    int pid;
    int arrival;
    int burst;
    int remaining;
    int completion;
    int turnaround;
    int waiting;
    char type[10];
};
int main() {
    struct Process SQ[MAX],UQ[MAX],all[MAX];
    int n,i,sq_count=0,uq_count=0;
    int time=0,completed=0;
    int TQ;
    printf("Enter number of processes: ");
    scanf("%d",&n);
    printf("Enter Time Quantum for System Queue: ");
    scanf("%d",&TQ);
    for (i = 0; i < n; i++) {
        printf("\nProcess %d\n", i + 1);
        all[i].pid = i + 1;
```

```

printf("Arrival Time: ");
scanf("%d", &all[i].arrival);
printf("Burst Time: ");
scanf("%d", &all[i].burst);
all[i].remaining = all[i].burst;
printf("Type (System/User): ");
scanf("%s", all[i].type);
if (strcmp(all[i].type, "System") == 0)
    SQ[sq_count++] = all[i];
else
    UQ[uq_count++] = all[i];
}
for (i = 0; i < uq_count - 1; i++) {
    for (int j = i + 1; j < uq_count; j++) {
        if (UQ[i].arrival > UQ[j].arrival) {
            struct Process temp = UQ[i];
            UQ[i] = UQ[j];
            UQ[j] = temp;
        }
    }
}
while (completed < n) {
    int executed = 0;
    for (i = 0; i < sq_count; i++) {
        if (SQ[i].remaining > 0 && SQ[i].arrival <= time) {
            executed = 1;
            if (SQ[i].remaining > TQ) {
                time += TQ;
                SQ[i].remaining -= TQ;
            } else {
                time += SQ[i].remaining;
            }
        }
    }
}

```

```

        SQ[i].remaining = 0;
        SQ[i].completion = time;
        SQ[i].turnaround = SQ[i].completion - SQ[i].arrival;
        SQ[i].waiting = SQ[i].turnaround - SQ[i].burst;
        all[SQ[i].pid - 1] = SQ[i];
        completed++;
    }
}
}
if (!executed) {
    for (i = 0; i < uq_count; i++) {
        if (UQ[i].remaining > 0 && UQ[i].arrival <= time) {
            executed = 1;
            time += UQ[i].remaining;
            UQ[i].remaining = 0;
            UQ[i].completion = time;
            UQ[i].turnaround = UQ[i].completion - UQ[i].arrival;
            UQ[i].waiting = UQ[i].turnaround - UQ[i].burst;
            all[UQ[i].pid - 1] = UQ[i];
            completed++;
            break;
        }
    }
}
if (!executed)
    time++;
}
float total_wt = 0, total_tat = 0;
printf("\nPID\tType\tAT\tBT\tCT\tTAT\tWT\n");
for (i = 0; i < n; i++) {
    printf("P%d\t%s\t%d\t%d\t%d\t%d\t%d\n",

```

```

        all[i].pid,
        all[i].type,
        all[i].arrival,
        all[i].burst,
        all[i].completion,
        all[i].turnaround,
        all[i].waiting);
    total_wt += all[i].waiting;
    total_tat += all[i].turnaround;
}
printf("\nAverage Waiting Time = %.2f", total_wt / n);
printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);
return 0;
}

```

Output:

```

Enter number of processes: 4
Enter Time Quantum for System Queue: 2

Process 1
Arrival Time: 0
Burst Time: 4
Type (System/User): 1

Process 2
Arrival Time: 0
Burst Time: 3
Type (System/User): 1

Process 3
Arrival Time: 0
Burst Time: 8
Type (System/User): 2

Process 4
Arrival Time: 10
Burst Time: 5
Type (System/User): 1

PID    Type    AT    BT    CT    TAT    WT
P1      1        0     4     4     4     0
P2      1        0     3     7     7     4
P3      2        0     8    15    15     7
P4      1       10     5    20    10     5

Average Waiting Time = 4.00
Average Turnaround Time = 9.00
1BF24CS327

```

Program 4:

Write a C program to simulate Real-Time CPU Scheduling algorithms:

- a)Rate- Monotonic
- b)Earliest-deadline First
- c)Proportional scheduling

Code:

=>Rate-Monotonic

```
#include <stdio.h>
#define MAX 10
int gcd(int a, int b) {
    while (b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}
int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}
int find_lcm(int arr[], int n) {
    int result = arr[0];
    for (int i = 1; i < n; i++) {
        result = lcm(result, arr[i]);
    }
    return result;
}
int main() {
    int n;
    int C[MAX], P[MAX];
    int remaining[MAX] = {0};
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter CPU burst times:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &C[i]);
    }
    printf("Enter time periods:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &P[i]);
    }
    for (int i = 0; i < n - 1; i++) {
```

```

    for (int j = i + 1; j < n; j++) {
        if (P[i] > P[j]) {
            int temp;
            temp = P[i]; P[i] = P[j]; P[j] = temp;
            temp = C[i]; C[i] = C[j]; C[j] = temp;
        }
    }
}
int hyperperiod = find_lcm(P, n);
printf("\nLCM = %d\n", hyperperiod);
printf("\nScheduling Timeline:\n");
for (int t = 0; t < hyperperiod; t++) {
    for (int i = 0; i < n; i++) {
        if (t % P[i] == 0) {
            remaining[i] = C[i];
        }
    }
    int selected = -1;
    for (int i = 0; i < n; i++) {
        if (remaining[i] > 0) {
            selected = i;
            break;
        }
    }
    if (selected != -1) {
        printf("%d ms: Process %d running\n", t, selected + 1);
        remaining[selected]--;
    } else {
        printf("%d ms: CPU is idle\n", t);
    }
}
printf("USN:1BF24CS327");
return 0;
}

```

Output:

```
C:\Users\BMSCECSE-L4\Desktop x + v
USN:1BF24CS327
Enter number of processes: 2
Enter CPU burst times:
20 35
Enter time periods:
50 100

LCM = 100
USN:1BF24CS327
Scheduling Timeline:
0 ms: Process 1 running
1 ms: Process 1 running
2 ms: Process 1 running
3 ms: Process 1 running
4 ms: Process 1 running
5 ms: Process 1 running
6 ms: Process 1 running
7 ms: Process 1 running
8 ms: Process 1 running
9 ms: Process 1 running
10 ms: Process 1 running
11 ms: Process 1 running
12 ms: Process 1 running
13 ms: Process 1 running
14 ms: Process 1 running
15 ms: Process 1 running
16 ms: Process 1 running
17 ms: Process 1 running
18 ms: Process 1 running
19 ms: Process 1 running
```

=>Earliest-deadline first:

```
#include <stdio.h>
```

```
#define MAX 20
```

```
typedef struct {
```

```
    int id;
```

```
    int arrival;
```

```
    int burst;
```

```
    int deadline;
```

```
    int completed;
```

```
} Process;
```

```
void edfScheduling(Process p[], int n) {
```

```
    int time = 0, completed = 0;
```

```
    printf("\n--- Earliest Deadline First Scheduling ---\n");
```

```
    while (completed < n) {
```

```
        int idx = -1;
```

```
        int minDeadline = 9999;
```

```
        for (int i = 0; i < n; i++) {
```

```
            if (!p[i].completed && p[i].arrival <= time) {
```

```
                if (p[i].deadline < minDeadline) {
```

```
                    minDeadline = p[i].deadline;
```

```
                    idx = i;
```

```
                }
```

```
            }
```

```
        }
```

```

    }
    if (idx != -1) {
        printf("Time %d -> %d : Process P%d\n",
            time, time + p[idx].burst, p[idx].id);
        time += p[idx].burst;
        p[idx].completed = 1;
        completed++;
    } else {
        time++;
    }
}
}
int main() {
    int n;
    Process p[MAX];
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("\nProcess %d\n", i + 1);
        p[i].id = i + 1;
        printf("Arrival Time: ");
        scanf("%d", &p[i].arrival);
        printf("Burst Time: ");
        scanf("%d", &p[i].burst);
        printf("Deadline: ");
        scanf("%d", &p[i].deadline);
        p[i].completed = 0;
    }
    edfScheduling(p, n);
    printf("USN:1BF24CS327");
    return 0;
}

```

Output:

```

Enter number of processes: 3

Process 1
Arrival Time: 2
Burst Time: 3
Deadline: 7

Process 2
Arrival Time: 3
Burst Time: 2
Deadline: 4

Process 3
Arrival Time: 2
Burst Time: 2
Deadline: 8

--- Earliest Deadline First Scheduling ---
Time 2 -> 5 : Process P1
Time 5 -> 7 : Process P2
Time 7 -> 9 : Process P3
USN:1BF24CS327

```

=>Proportional Scheduling:

```

#include <stdio.h>
#define MAX 20
typedef struct {
    int id;
    int weight;
} Process;
void proportionalScheduling(Process p[], int n, int total_time) {
    int total_weight = 0;
    printf("\n--- Proportional Scheduling ---\n");
    for (int i = 0; i < n; i++) {
        total_weight += p[i].weight;
    }
    for (int i = 0; i < n; i++) {
        float cpu_share = ((float)p[i].weight / total_weight) * total_time;
        printf("Process P%d gets CPU time = %.2f units\n",
            p[i].id, cpu_share);
    }
}
int main() {
    int n, total_time;
    Process p[MAX];
    printf("Enter number of processes: ");
    scanf("%d", &n);

```

```

for (int i = 0; i < n; i++) {
    printf("\nProcess %d\n", i + 1);
    p[i].id = i + 1;
    printf("Enter weight: ");
    scanf("%d", &p[i].weight);
}
printf("\nEnter total CPU time: ");
scanf("%d", &total_time);
proportionalScheduling(p, n, total_time);
return 0;
}

```

Output:

```

1BF24CS327
Enter number of processes: 3

Process 1
Enter weight: 10

Process 2
Enter weight: 20

Process 3
Enter weight: 30

Enter total CPU time: 5

--- Proportional Scheduling ---
Process P1 gets CPU time = 0.83 units
Process P2 gets CPU time = 1.67 units
Process P3 gets CPU time = 2.50 units

```

Program 5:

Write a C program to simulate producer-consumer problem using semaphores.

Code:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <semaphore.h>
#define BUFFER_SIZE 5
int buffer[BUFFER_SIZE];
int in = 0, out = 0;
sem_t empty; // counts empty slots
sem_t full; // counts filled slots
pthread_mutex_t mutex; // for mutual exclusion
void* producer(void* arg) {
    int item, i = 0;
    while (1) {
        item = i++; // produce an item (here just incrementing number)
        sem_wait(&empty); // wait for empty slot
        pthread_mutex_lock(&mutex); // enter critical section
        buffer[in] = item;
        printf("Produced: %d at buffer[%d]\n", item, in);
        in = (in + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex); // leave critical section
        sem_post(&full); // signal that new item is available
        sleep(1); // simulate time to produce
    }
}
void* consumer(void* arg)
{
    int item;
    while (1)
    {
        sem_wait(&full); // wait for filled slot
        pthread_mutex_lock(&mutex); // enter critical section
        item = buffer[out];
        printf("Consumed: %d from buffer[%d]\n", item, out);
        out = (out + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex); // leave critical section
        sem_post(&empty); // signal that a slot is free
        sleep(2); // simulate time to consume
    }
}
```

```

}
}
int main()
{
pthread_t prod, cons;
sem_init(&empty, 0, BUFFER_SIZE);
sem_init(&full, 0, 0); pthread_mutex_init(&mutex, NULL);
pthread_create(&prod, NULL, producer, NULL);
pthread_create(&cons, NULL, consumer, NULL);
pthread_join(prod, NULL);
pthread_join(cons, NULL);
sem_destroy(&empty);
sem_destroy(&full);
pthread_mutex_destroy(&mutex);
return 0;

}

```

Output:

```

1BF24CS327
Produced: 0 at buffer[0]
Consumed: 0 from buffer[0]
Produced: 1 at buffer[1]
Consumed: 1 from buffer[1]
Produced: 2 at buffer[2]
Produced: 3 at buffer[3]
Consumed: 2 from buffer[2]
Produced: 4 at buffer[4]
Produced: 5 at buffer[0]
Consumed: 3 from buffer[3]
Produced: 6 at buffer[1]
Produced: 7 at buffer[2]
Consumed: 4 from buffer[4]
Produced: 8 at buffer[3]
Produced: 9 at buffer[4]
Consumed: 5 from buffer[0]
Produced: 10 at buffer[0]

```

Program 6:

Write a C program to simulate the concept of Dining Philosophers problem.

Code:

```
#include<stdio.h>
#include<pthread.h>
#include<unistd.h>
#define N 5 // Number of philosophers
pthread_mutex_t forks[N]; // One mutex per fork
pthread_t philosophers[N];
void* philosopher(void* num)
{
    int id = *(int*)num;
    int left = id; // left fork index
    int right = (id + 1) % N; // right fork index
    while (1)
    {
        printf("Philosopher %d is thinking.\n", id);
        sleep(1); // Thinking
        // To avoid deadlock, philosophers with even id pick left then right,
        // odd id pick right then left.
        if (id % 2 == 0)
        {
            // Pick up left fork
            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up left fork %d.\n", id, left);
            // Pick up right fork
            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id, right); }
        else
        { // Pick up right fork
            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id, right); // Pick up left
            fork pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up left fork %d.\n", id, left);
        }
        // Eating
        printf("Philosopher %d is eating.\n", id);
        sleep(2); // Put down forks
        pthread_mutex_unlock(&forks[left]);
        pthread_mutex_unlock(&forks[right]);
    }
}
```

```

printf("Philosopher %d put down forks %d and %d.\n", id, left, right);
}
return NULL;
}
int main() {
int i;
int ids[N]; // Initialize forks (mutexes)
for (i = 0; i < N; i++)
{ pthread_mutex_init(&forks[i], NULL);
ids[i] = i;
}
// Create philosopher threads
for (i = 0; i < N; i++)
{
pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
} // Join threads (will never happen here, but good practice)
for (i = 0; i < N; i++)
{
pthread_join(philosophers[i], NULL); }
// Destroy mutexes (unreachable here)
for (i = 0; i < N; i++)
{
pthread_mutex_destroy(&forks[i]);
}
return 0;
}

```

Output:

```

1BF24CS327
Philosopher 0 is thinking.
Philosopher 1 is thinking.
Philosopher 2 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 3 picked up right fork 4.
Philosopher 3 picked up left fork 3.
Philosopher 3 is eating.
Philosopher 1 picked up right fork 2.
Philosopher 0 put down forks 0 and 1.
Philosopher 0 is thinking.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 3 put down forks 3 and 4.

```

Program 7:

Write a C program to simulate Bankers algorithm for the purpose of deadlock avoidance.

Code:

```
#include <stdio.h>
#define MAX 10
int main() {
    int n, m, i, j, k;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resources: ");
    scanf("%d", &m);
    int alloc[MAX][MAX], max[MAX][MAX], need[MAX][MAX];
    int avail[MAX];
    printf("Enter Allocation Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }
    printf("Enter Maximum Demand Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            scanf("%d", &max[i][j]);
        }
    }
    printf("Enter Available Resources:\n");
    for(i = 0; i < m; i++) {
        scanf("%d", &avail[i]);
    }
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }
    int finish[MAX] = {0};
    int safeSeq[MAX];
    int work[MAX];
    for(i = 0; i < m; i++) {
        work[i] = avail[i];
    }
    int count = 0;
```

```

while(count < n) {
    int found = 0;
    for(i = 0; i < n; i++) {
        if(finish[i] == 0) {
            for(j = 0; j < m; j++) {
                if(need[i][j] > work[j])
                    break;
            }
            if(j == m) {
                for(k = 0; k < m; k++) {
                    work[k] += alloc[i][k];
                }
                safeSeq[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }
    if(found == 0) {
        break;
    }
}
if(count == n) {
    printf("\nSystem is in a safe state.\n");
    printf("Safe sequence is: ");
    for(i = 0; i < n; i++) {
        printf("P%d", safeSeq[i]);
        if(i != n - 1)
            printf(" -> ");
    }
} else {
    printf("\nSystem is NOT in a safe state.\n");
}
return 0;
}

```

Output:

```
1BF24CS327
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum Demand Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2

System is in a safe state.
Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2
```